

INTERVIEW PREPARATION GUIDE

Playwright Automation, TypeScript & JavaScript

Total Questions: 75

Based on course material by Bharath Reddy

Prepared for: Student Interview Preparation

TABLE OF CONTENTS

SECTION A: PLAYWRIGHT AUTOMATION – KEY TERMINOLOGY

- Q1. What is a Requirement in software testing?
- Q2. What is a Requirement Document?
- Q3. What is the difference between a Project and a Product?
- Q4. What is an Environment in software testing?
- Q5. What is a Build in software testing?
- Q6. What is a Release in software testing?
- Q7. What are Release Notes?
- Q8. What is QA Sign-off?
- Q9. Why is Node.js required for Playwright?
- Q10. What is an IDE and why do we use VS Code for Playwright?
- Q11. What is an npm package?
- Q12. What is the difference between npm and npx?
- Q13. What is a Playwright Browser?
- Q14. What is a Browser Context in Playwright?
- Q15. What is a Page in Playwright?
- Q16. What is a Worker in Playwright?
- Q17. What are Assertions in Playwright?
- Q18. What is the difference between Headed and Headless mode in Playwright?
- Q19. What is a Trace in Playwright?
- Q20. What is a Framework in automation testing?
- Q21. What is BDD (Behavior Driven Development)?
- Q22. What is Page Object Model (POM)?
- Q23. What is CI/CD and a CI/CD Pipeline?
- Q24. What is Source Code Management (SCM)?
- Q25. What is Copilot and how is it used in Playwright automation?
- Q26. What is MCP (Model Context Protocol)?
- Q27. What is an LLM (Large Language Model)?

SECTION B: PLAYWRIGHT NOTES – CORE CONCEPTS

- Q28. What is Playwright?
- Q29. What are the key differences between Playwright and Selenium?
- Q30. How do you set up a Playwright project?
- Q31. Explain the Playwright folder structure.
- Q32. What are the key configurations in playwright.config.ts?
- Q33. Explain Playwright's parallel execution model.
- Q34. List important Playwright run commands.
- Q35. What are Playwright Annotations? List them.
- Q36. What is the difference between test.skip() and test.fixme()?
- Q37. What are Playwright Tags and how do you use them?
- Q38. What are Playwright Hooks? Explain the different types.

- Q39. Explain Playwright's Browser Architecture.
- Q40. Explain the different types of Assertions in Playwright.
- Q41. What is the difference between Hard and Soft Assertions?
- Q42. What are Playwright Locators? List the types and priority order.
- Q43. What are the common Web Actions in Playwright?
- Q44. How does Playwright handle alerts and dialogs?
- Q45. How does Playwright handle iframes?
- Q46. How do you upload files in Playwright?

SECTION C: TYPESCRIPT / JAVASCRIPT FUNDAMENTALS

- Q47. What is JavaScript?
- Q48. What is TypeScript and why is it preferred over JavaScript?
- Q49. What are the common print statements in JavaScript/TypeScript?
- Q50. What are the different ways to declare variables in JavaScript/TypeScript?
- Q51. What is Block Scope in JavaScript?
- Q52. What are Primitive Data Types in JavaScript/TypeScript?
- Q53. What are Non-Primitive (Reference) Data Types in JavaScript/TypeScript?
- Q54. What is a Union Type in TypeScript?
- Q55. What is a Tuple in TypeScript?
- Q56. Explain the different types of Operators in TypeScript.
- Q57. What is the difference between == and === in JavaScript/TypeScript?
- Q58. Explain if/else and switch/case conditional statements.
- Q59. What are the different types of loops in TypeScript?
- Q60. What are Jumping Statements in TypeScript?
- Q61. What are the different types of Functions in TypeScript?
- Q62. What are the different function implementations in TypeScript?
- Q63. Explain Object methods and operations in TypeScript.
- Q64. What are the common Array methods in JavaScript/TypeScript?
- Q65. What are common String methods in JavaScript/TypeScript?
- Q66. How do you perform Data Conversions in JavaScript/TypeScript?
- Q67. What are Modules in TypeScript? Explain imports and exports.
- Q68. Explain Exception Handling in JavaScript/TypeScript.
- Q69. What are Classes, Objects, and Constructors in TypeScript?
- Q70. What are Access Modifiers in TypeScript?
- Q71. What is the difference between 'this' and 'super' keywords?
- Q72. Explain Inheritance in TypeScript.
- Q73. Explain the four OOP concepts: Abstraction, Encapsulation, Polymorphism, and Inheritance.
- Q74. What are Static Properties and Methods in TypeScript?
- Q75. Explain async/await, Promises, and Callback functions.

SECTION A: PLAYWRIGHT AUTOMATION – KEY TERMINOLOGY

Q1. What is a Requirement in software testing?

Answer: A requirement is a description of the services that a software system must provide and the conditions under which it must operate. In simple terms, it represents the client's expectation from the product. For example, in a CRM project, requirements may cover customer interaction management related to sales, support, and marketing.

Q2. What is a Requirement Document?

Answer: A requirement document is a document or set of documentation that describes the features and behavior of a system or software application. It includes elements that define the intended functionality required by the customer to satisfy their users. For example, a BRD (Business Requirements Document) for a CRM Services project.

Q3. What is the difference between a Project and a Product?

Answer: A Project is something developed based on a particular customer's requirements – it is a specific effort to create or modify a product or service. A Product is an application, system, or platform developed based on company/client specifications and used by multiple customers. For example, Creatio CRM is the product, while implementing Case & Services modules for a specific client is the project.

Q4. What is an Environment in software testing?

Answer: An environment is a configured setup where the software application is deployed, executed, and tested. It includes hardware, operating system, application servers, databases, network settings, third-party integrations, and configuration values. In simple terms, it is 'where the application lives and runs' at different stages of development, testing, and delivery. Common environments include QA, UAT, Staging, and Production.

Q5. What is a Build in software testing?

Answer: A build is a version of software that is compiled and ready for testing. Each build includes new features, bug fixes, and other updates since the previous build. Builds are typically identified by a Build ID (e.g., v1.0.23-QA), deployed to specific environments, and tested by the QA team.

Q6. What is a Release in software testing?

Answer: A release is the process of delivering a final build of software to end-users. A release may include multiple builds and is often accompanied by release notes detailing new

features, fixes, and known issues. Releases go through formal approval and sign-off processes.

Q7. What are Release Notes?

Answer: Release notes are documents prepared by the dev team during release time, delivered to customers containing technical information about changes addressed in the current release. They typically include the objective of the release, features delivered, development details, QA validation summary, fixes addressed, known issues, and deployment details.

Q8. What is QA Sign-off?

Answer: QA Sign-off is the formal approval given by the QA team to declare that a particular build, feature, or release is tested thoroughly, meets acceptance criteria, and is ready for deployment to the next environment (UAT, staging, or production). It indicates QA confidence in the quality and stability of the application.

Q9. Why is Node.js required for Playwright?

Answer: Node.js environment is the runtime setup that allows JavaScript and TypeScript code to execute outside the browser. Playwright is built on Node.js, and test execution, browser control, and reporting all run inside Node. npm/npx commands required by Playwright only work with Node.js installed.

Q10. What is an IDE and why do we use VS Code for Playwright?

Answer: An IDE (Integrated Development Environment) is a software application that provides tools to write, edit, run, debug, and manage code. VS Code (Visual Studio Code) is a lightweight yet powerful IDE used in Playwright development to write test scripts, create Page Object Models, run tests from the terminal, debug failures, and integrate with Git and Copilot.

Q11. What is an npm package?

Answer: An npm package is a reusable library or tool published to the Node Package Manager (npm) registry. Each package contains JavaScript/TypeScript code, dependencies, version information, and optionally executable commands. In Playwright, we install @playwright/test as an npm package which includes the test runner, browser automation APIs, assertions, and configuration support.

Q12. What is the difference between npm and npx?

Answer: npm (Node Package Manager) is used to install and manage packages. npx is used to execute packages directly without installing them globally. For example, 'npm install @playwright/test' installs Playwright, while 'npx playwright test' executes the test runner. npx looks for the package in node_modules first; if not found, it temporarily downloads it, executes the command, and cleans up.

Q13. What is a Playwright Browser?

Answer: A Playwright browser is the actual browser engine that Playwright controls to execute tests. Playwright supports three browser engines: Chromium (Chrome, Edge), Firefox, and WebKit (Safari-like). Unlike Selenium, Playwright communicates directly with browser engines and does not require separate driver installations.

Q14. What is a Browser Context in Playwright?

Answer: A browser context is an isolated browser session within a single browser instance, similar to a new incognito window. Each context has separate cookies, local/session storage, and separate cache. Browser contexts are used to simulate multiple users, run tests independently, and avoid test data collision.

Q15. What is a Page in Playwright?

Answer: A Page represents a single browser tab or window. It is where we perform actions like click, type, and validate UI. Each screen in the application (e.g., Login, Dashboard, Case Details) is handled using a Page object. Example: `await page.goto('/Login');`

Q16. What is a Worker in Playwright?

Answer: A worker is an independent Playwright process used to run tests in parallel. Workers are parallel execution engines. Configuring multiple workers (e.g., `workers: 4`) means 4 tests can run at the same time, reducing overall execution time and running the regression suite faster.

Q17. What are Assertions in Playwright?

Answer: An assertion is a verification step that checks whether the application behaves as expected. Assertions decide pass or fail. Playwright assertions are used to validate page titles, UI text, element visibility, and API responses. Example: `expect(page.locator('h1')).toHaveText('Cases');` If an assertion fails, the test fails.

Q18. What is the difference between Headed and Headless mode in Playwright?

Answer: Headless mode runs the browser without displaying the UI – the browser runs in the background. It is the default mode, used in CI/CD pipelines (Jenkins, Azure) for faster execution on servers without GUI. Headed mode runs the browser with visible UI, allowing us to see the browser while the test runs. It is used for debugging flaky tests, observing UI behavior, and validating animations. Command: `npx playwright test --headed`.

Q19. What is a Trace in Playwright?

Answer: A trace is a complete execution recording of a Playwright test. It captures screenshots, DOM snapshots, network calls, user actions, and console logs. Tracing is used

to analyze failed tests, debug CI failures, and share issues with developers. It can be enabled with: `npx playwright test --trace=on`.

Q20. What is a Framework in automation testing?

Answer: A framework is a pre-packaged set of solutions to solve common development problems, providing a foundation and tools for developers to create applications more efficiently. Without a framework, scripts are messy, hard to reuse, tightly coupled with test data, and maintenance becomes difficult. A good framework provides reusability of code, better readability, standardized reporting, and faster execution. Key components include driver scripts, test scripts, object repository, reports and logs, test data, config files, utilities, and assertion engines.

Q21. What is BDD (Behavior Driven Development)?

Answer: BDD is a software development approach that emphasizes collaboration between developers, testers, and business stakeholders. It uses natural language (Given-When-Then) to write test scenarios, bridging the gap between technical implementation and business expectations. Key concepts include Feature Files (containing test scenarios in Gherkin syntax), Scenarios, Step Definitions, Runner Class, Glue Code, and Gherkin Language.

Q22. What is Page Object Model (POM)?

Answer: POM is a design pattern in software testing where each web page in an application is represented by a separate class, with methods to interact with the page's elements. Key benefits include: Reusability – code for page interaction is written once; Clean Separation – test logic is separate from page structure; Easy Maintenance – UI changes affect only the page class, not test scripts; Scalability – perfect for large applications with many pages.

Q23. What is CI/CD and a CI/CD Pipeline?

Answer: CI/CD stands for Continuous Integration and Continuous Deployment/Delivery. It is a process where code changes are automatically built, tested, and deployed to production. A CI/CD pipeline is the series of steps code changes go through: detecting code changes, triggering the test automation framework, downloading dependencies, executing tests (Smoke, Regression), generating reports, and sending results via email/Slack/dashboards. Components include SCM (GitHub/GitLab), CI/CD tools (Jenkins/GitHub Actions), build tools, test frameworks, reports, and notification systems.

Q24. What is Source Code Management (SCM)?

Answer: SCM refers to tools and processes used to manage code versions, track changes, and enable collaboration among developers and QA teams. It ensures changes are tracked, code is version-controlled, and multiple people can work safely and simultaneously. Benefits include version control, collaboration, rollback capability, audit trails, and CI/CD integration. Common Git terms include Repository, Commit, Branch, Merge, Pull Request, Clone, and Push/Pull.

Q25. What is Copilot and how is it used in Playwright automation?

Answer: Copilot is an AI-powered coding assistant that helps developers write code faster and with fewer errors by suggesting code, explanations, and fixes directly inside the IDE. It can suggest code line-by-line, auto-complete functions, generate test cases, explain existing code, and refactor logic. In Playwright automation, Copilot is used in VS Code to generate test cases, suggest locators and assertions, create Page Object methods, and speed up repetitive automation tasks.

Q26. What is MCP (Model Context Protocol)?

Answer: MCP is a standardized way for AI models to securely interact with external tools and systems like browsers, files, databases, and APIs. It acts as a bridge between the AI brain and real-world tools. In Playwright automation, using MCP, AI can open a browser, navigate to the application, locate elements, generate Playwright scripts, and save files automatically. The flow is: AI Prompt → MCP → Playwright → Browser → Test Code.

Q27. What is an LLM (Large Language Model)?

Answer: An LLM is an AI model trained on massive amounts of text data to understand, generate, and reason using human language. LLMs can understand requirements, generate code, explain concepts, answer questions, and analyze logs and errors. In automation, LLMs are used to understand BRD/requirements, generate test scenarios, suggest edge cases, explain Playwright failures, and assist in framework design.

SECTION B: PLAYWRIGHT NOTES – CORE CONCEPTS

Q28. What is Playwright?

Answer: Playwright is a modern, open-source end-to-end test automation framework used for testing web applications. It allows you to automate user actions like clicking buttons, filling forms, navigating pages, and validating UI behavior across different browsers. Playwright supports Chromium, Firefox, and WebKit, so you can test on Chrome, Firefox, Safari, and Edge. It runs fast, handles modern web features well, and is designed to test real user interactions in a reliable and efficient way.

Q29. What are the key differences between Playwright and Selenium?

Answer: 1) Architecture: Selenium uses client-server architecture with WebDriver; Playwright communicates directly with browsers, making it faster. 2) Browser Support: Selenium needs separate drivers (ChromeDriver, GeckoDriver); Playwright supports Chromium, Firefox, WebKit out of the box. 3) Speed & Stability: Selenium can be slower and flaky; Playwright is faster with built-in auto-waiting. 4) Modern Web Handling: Selenium struggles with SPAs; Playwright is built for modern web apps (React, Angular, Vue). 5) Language Support: Selenium supports Java, Python, C#, JS, Ruby; Playwright focuses on JS/TS with support for Java, Python, .NET. 6) Playwright has built-in reports, API testing, screenshots/videos, and parallel execution; Selenium requires external libraries.

Q30. How do you set up a Playwright project?

Answer: Step 1: Create a project folder. Step 2: Change directory to the project folder. Step 3: Run 'npm init playwright@latest'. Step 4: Select TypeScript. Step 5: Choose the test directory (default: tests). Step 6: Skip GitHub Actions workflow (press n). Step 7: Install Playwright browsers (press Y) and wait for setup to complete. Step 8: Verify the folder structure.

Q31. Explain the Playwright folder structure.

Answer: The main folders/files are: 1) node_modules/ – contains all installed dependencies (auto-generated, never edit manually, not committed to Git). 2) tests/ – where all test cases live; Playwright automatically scans this folder. 3) example.spec.ts – a sample test file written in TypeScript (must end with .spec.ts or .test.ts). 4) playwright.config.ts – central configuration file controlling browser selection, modes, base URL, timeouts, reports, parallel execution. 5) package.json – project metadata and scripts. 6) package-lock.json – locks exact dependency versions for consistent setup across machines.

Q32. What are the key configurations in playwright.config.ts?

Answer: Key configurations include: `testDir` (specifies where test files are located), `fullyParallel` (enables parallel execution of test files), `forbidOnly` (prevents accidental commits of test.only – fails build on CI), `retries` (defines retry count for failed tests), `workers` (controls number of parallel workers), `reporter` (e.g., 'html' for HTML reports), `trace` (collects screenshots, DOM snapshots, network calls, console logs), and `projects` (configures multiple browsers like Chromium, Firefox, WebKit).

Q33. Explain Playwright's parallel execution model.

Answer: Playwright parallelism depends on two things: `workers` (controls spec file parallelism) and `fullyParallel` (controls test-level parallelism). Case 1: `workers:1`, `fullyParallel:false` – everything runs sequentially (slowest, most stable). Case 2: `workers:1`, `fullyParallel:true` – spec files sequential, tests inside parallel. Case 3: `workers:2`, `fullyParallel:false` – spec files parallel, tests inside sequential (BEST PRACTICE). Case 4: `workers:2`, `fullyParallel:true` – everything parallel (fastest but highest flakiness risk). `test.describe.parallel()` can override `fullyParallel:false` for specific test blocks.

Q34. List important Playwright run commands.

Answer: `npm init playwright@latest` – install Playwright. `npx playwright test` – run all tests. `npx playwright test tests/file.spec.ts` – run specific file. `npx playwright test -g "test name"` – run specific test. `npx playwright test --headed` – run with browser UI. `npx playwright test --project=chromium` – run in specific browser. `npx playwright show-report` – view HTML report. `npx playwright test --trace=on` – record trace. `npx playwright test --debug` – debug mode. `npx playwright test --ui` – UI mode. `npx playwright test --last-failed` – run last failed tests. `npx playwright codegen URL` – record and generate test script.

Q35. What are Playwright Annotations? List them.

Answer: Playwright annotations are special helpers to control how tests are grouped, executed, skipped, or marked. They include: `test.describe()` – groups related tests for readability and organization; `test.only()` – runs only the selected test (should never be committed to version control); `test.skip()` – skips a test intentionally; `test.fixme()` – marks a test as broken/to be fixed later (unlike skip, it signals a known issue requiring action); `test.slow()` – increases test timeout by 3x (default 30s becomes 90s, does NOT slow execution); `test.fail()` – marks a test as expected to fail (if it fails → reported as passed; if it passes → reported as failed).

Q36. What is the difference between `test.skip()` and `test.fixme()`?

Answer: `test.skip()` is an intentional skip used when a feature is not ready or there are environment limitations – it has a neutral status. `test.fixme()` is used for a known broken test where a bug exists – it indicates action is required. Both prevent the test from executing, but `fixme` clearly flags it as a known issue needing a fix.

Q37. What are Playwright Tags and how do you use them?

Answer: Tags are labels assigned to tests for selective execution. They help separate smoke, sanity, and regression tests, control CI/CD execution, and manage large test suites. Single tag syntax: `test('Test name', { tag: '@smoke' }, async () => {})`. Multiple tags: `test('Test name', { tag: ['@smoke', '@regression'] }, async () => {})`. Run tagged tests with: `npx playwright test --grep "@smoke"`. Run multiple tags: `npx playwright test --grep "@smoke|@regression"`.

Q38. What are Playwright Hooks? Explain the different types.

Answer: Hooks are special functions that run before or after tests for setup and cleanup. They reduce duplicate code, manage preconditions, clean up after tests, and maintain consistent environments. Types: `beforeAll` – runs once before all tests (one-time setup, DB connection, auth); `afterAll` – runs once after all tests (closing connections, cleaning data); `beforeEach` – runs before every test (launching app, navigating to base URL); `afterEach` – runs after every test (logging results, cleaning cookies, taking screenshots). Hooks can be global-level (outside `test.describe`, apply to all tests) or group-level (inside `test.describe`, apply only within that group).

Q39. Explain Playwright's Browser Architecture.

Answer: Playwright follows a three-layer model: 1) Browser – the actual browser engine (Chrome, Firefox, WebKit). 2) Browser Context – an isolated environment for cookies, cache, and session (like an incognito window). 3) Page – represents a browser tab or window. The hierarchy is: Browser → Context → Page(s). Unlike Selenium, Playwright does not rely on window handles and uses this layered architecture for isolation and parallel execution.

Q40. Explain the different types of Assertions in Playwright.

Answer: Playwright assertions use the `expect()` API and are auto-retrying (default timeout: 5 seconds). Types include: 1) Page-level: `toHaveTitle()`, `toHaveURL()`. 2) Visibility/State: `toBeVisible()`, `toBeEnabled()`, `toBeDisabled()`. 3) Text: `toHaveText()` (exact match), `toContainText()` (partial match). 4) Value: `toHaveValue()` for input fields. 5) Attribute: `toHaveAttribute()`. 6) Form Control: `toBeChecked()` for checkboxes/radio buttons. 7) Negation: `not.toBeVisible()`, `not.toBeChecked()`, etc. 8) Soft Assertions: `expect.soft()` continues execution even on failure; all failures reported together at end.

Q41. What is the difference between Hard and Soft Assertions?

Answer: Hard Assertions (default `expect()`) stop the test immediately on failure and are used for critical validations. Soft Assertions (`expect.soft()`) allow the test to continue execution even if an assertion fails – all soft assertion failures are reported together at the end. Soft assertions are useful for UI validation checks and multiple independent validations.

Q42. What are Playwright Locators? List the types and priority order.

Answer: Locators are used to identify and interact with web elements. They are lazy, auto-waiting, and re-evaluated on each action. Built-in (recommended) locators in priority order: 1) `getByRole()` – accessibility-based, most stable. 2) `getByLabel()` – for form controls by label text. 3) `getByText()` – by visible text content. 4) `getByPlaceholder()` – by placeholder attribute.

5) `getByAltText()` – for images by alt attribute. 6) `getByTitle()` – by title/tooltip attribute. 7) `getByTestId()` – by data-testid attribute, most stable for automation. Generic locators (less stable): CSS selectors via `page.locator('.class')` and XPath via `page.locator('//xpath')`.

Q43. What are the common Web Actions in Playwright?

Answer: Key web actions include: Navigation: `page.goto(URL)`, `page.waitForURL()`. Window: `page.setViewportSize()`. Cookies: `page.context().clearCookies()`. Button actions: `click()`, `dblclick()`, `click({button:'right'})`, `hover()`, `dragTo()`, `scrollIntoViewIfNeeded()`, `click({force:true})`. Textbox: `clear()`, `fill()`, `type()`, `keyboard.press()`. Dropdown: `selectOption()` by label/value/index. Checkbox: `check()`, `isChecked()`. File upload: `setInputFiles()`. Alert handling: `page.on('dialog', handler)`. Frame handling: `page.frameLocator('#id').locator()`. Best practices: prefer auto-waiting over manual waits, validate visibility before interaction, avoid force actions unless necessary.

Q44. How does Playwright handle alerts and dialogs?

Answer: Playwright handles alerts using the `page.on('dialog')` event listener. You register a handler that receives the dialog object, which provides methods like `dialog.message()` to get the alert text, `dialog.accept()` to click OK, and `dialog.dismiss()` to click Cancel. This handles JavaScript alerts, confirm dialogs, and prompt dialogs.

Q45. How does Playwright handle iframes?

Answer: Playwright uses `page.frameLocator('#frameId')` to interact with elements inside iframes. You first create a frame locator using the iframe's selector, then use regular locator methods on it. Example: `const frame = page.frameLocator('#frameId'); await frame.locator('#element').click();`

Q46. How do you upload files in Playwright?

Answer: Playwright provides the `setInputFiles()` method for file uploads without interacting with OS dialogs. Example: `await upload.setInputFiles('tests/resources/file.txt')`. This directly sets the file on the input element.

SECTION C: TYPESCRIPT / JAVASCRIPT FUNDAMENTALS

Q47. What is JavaScript?

Answer: JavaScript is a versatile, object-oriented scripting programming language primarily used for making websites interactive. It runs in web browsers, helps create dynamic and engaging user experiences, allows you to implement complex features on web pages, create dynamically updating content, control multimedia, and animate images.

Q48. What is TypeScript and why is it preferred over JavaScript?

Answer: TypeScript is a strongly-typed, object-oriented superset of JavaScript used for building scalable and maintainable web applications. It adds static typing, better tooling, and early error detection, and compiles down to plain JavaScript. Problems with JavaScript: no type safety leads to runtime errors (e.g., `add(10, '5')` returns `'105'`). TypeScript fixes this with static typing that catches errors at compile time (e.g., `add(10, '5')` gives a compile-time error when parameters are typed as number).

Q49. What are the common print statements in JavaScript/TypeScript?

Answer: `console.log()` – general output; `console.info()` – informational messages; `console.warn()` – warning messages; `console.error()` – error messages. Use print statements for quick debugging, learning, prototyping, and real-time monitoring. Avoid in production code (clutters output, may expose sensitive info) and performance-critical code.

Q50. What are the different ways to declare variables in JavaScript/TypeScript?

Answer: There are three ways: `var` (legacy, from 1995-2015, function-scoped, can be redeclared), `let` (introduced in ES6/2015, block-scoped, cannot be redeclared), and `const` (introduced in ES6/2015, block-scoped, cannot be redeclared or reassigned). Best practice: always use `const` unless the value will change, then use `let`. Avoid `var` in modern code. TypeScript syntax adds type annotations: `let x: string = 'hello'; const y: number = 42;`

Q51. What is Block Scope in JavaScript?

Answer: Block scope means variables declared inside a `{ }` block cannot be accessed from outside. `let` and `const` have block scope. `var` does NOT have block scope – variables declared with `var` inside a block can be accessed outside it. Example: `{ let x = 2; } // x NOT accessible outside. { var x = 2; } // x IS accessible outside.` This is why `let` and `const` are preferred over `var`.

Q52. What are Primitive Data Types in JavaScript/TypeScript?

Answer: Primitive data types are simple, immutable values stored directly in memory and accessed by value. There are six: 1) number – integers and floating-point numbers. 2) string – text in single quotes, double quotes, or backticks. 3) boolean – true or false. 4) null – intentional absence of a value. 5) undefined – variable declared but not assigned. 6) symbol – unique identifiers (ES6). In TypeScript, types are explicitly declared: let marks: number = 98; let name: string = 'Bharath';

Q53. What are Non-Primitive (Reference) Data Types in JavaScript/TypeScript?

Answer: Non-primitive types are complex, mutable values stored by reference (memory address). They include: 1) Object – collection of key-value pairs. 2) Array – ordered list of values. 3) Function – reusable block of code. 4) Date – date and time values. 5) Map – key-value pairs with any data type as keys (ES6). 6) Set – collection of unique values (ES6). TypeScript also has: tuple (fixed-size array with specific types per position) and any (allows any type, should be avoided).

Q54. What is a Union Type in TypeScript?

Answer: A union type allows a variable to store multiple data types using the | operator. It is a type-safe alternative to 'any'. Example: let empAddress: string | number | boolean; empAddress = 'Bangalore'; empAddress = 560068; Both assignments are valid because the variable accepts string, number, or boolean.

Q55. What is a Tuple in TypeScript?

Answer: A tuple is a fixed-size array where each element has a specific data type and position. Characteristics: fixed length, order matters, each index has a predefined data type. Example: let address: [string, string, number] = ['221B Baker Street', 'London', 560068];

Q56. Explain the different types of Operators in TypeScript.

Answer: 1) Arithmetic: +, -, *, /, %, ++, -- (pre/post increment/decrement). 2) Assignment: =, +=, -=, *=, /=, %. 3) Comparison/Relational: == (loose equality, checks value), === (strict equality, checks value AND type), !=, !==, >, <, >=, <= . 4) Logical: && (AND – true only if both true), || (OR – false only if both false), ! (NOT – reverses boolean). 5) Ternary: condition ? value_if_true : value_if_false – shorthand for if-else.

Q57. What is the difference between == and === in JavaScript/TypeScript?

Answer: == is loose equality – it checks only the value and performs type coercion. === is strict equality – it checks both the value AND the type. Example: let x: number = 10; let y: any = '10'; x == y returns true (values match after coercion); x === y returns false (types are different: number vs string). Best practice: always use === for comparisons.

Q58. Explain if/else and switch/case conditional statements.

Answer: if/else: The if statement executes a block if a condition is true, else executes an alternative block. Supports else if for multiple conditions and nested if/else. switch/case: Used when checking multiple conditions against a single variable. Provides a cleaner alternative to a series of if/else if statements. Each case ends with break to prevent fall-through. The default case handles unmatched values.

Q59. What are the different types of loops in TypeScript?

Answer: 1) for loop – used when iterations are known: `for(let i=1; i<=10; i++)`. 2) while loop – continues while condition is true, iterations unknown. 3) do-while loop – like while but guarantees at least one execution. 4) for...in loop – iterates over object properties (keys). 5) for...of loop (ES6) – iterates over iterable values (arrays, strings, maps, sets).

Q60. What are Jumping Statements in TypeScript?

Answer: Jumping statements change the normal flow of execution. Types: 1) break – immediately exits a loop or switch statement. 2) continue – skips the current iteration and moves to the next. 3) return – exits a function and optionally sends a value back. 4) throw – stops execution by throwing an error (e.g., `throw new Error('Invalid amount')`).

Q61. What are the different types of Functions in TypeScript?

Answer: 1) Named Function – has an explicit name, declared with function keyword: `function greet(name: string): string { return 'Hello ' + name; }`. 2) Anonymous Function – no name, assigned to a variable: `const greet = function(name: string): string { return 'Hello ' + name; }`. 3) Arrow Function (Lambda) – shorter syntax using `=>`: `const add = (a: number, b: number): number => a + b`. Arrow functions were introduced in ES6 for cleaner, shorter code.

Q62. What are the different function implementations in TypeScript?

Answer: 1) Without Parameters & Without Return (void). 2) With Parameters & Without Return (void). 3) Without Parameters & With Return Type. 4) With Parameters & With Return Type. 5) Optional Parameters (`param?: type`) – parameters not required. 6) Default Parameters (`param: type = defaultValue`) – uses default if no argument passed. 7) Rest Parameters (`...param: type[]`) – accepts unknown number of arguments.

Q63. Explain Object methods and operations in TypeScript.

Answer: Objects are key-value pairs created with `{}`. Operations include: Creating with interface for type safety, accessing properties via dot notation (`obj.property`) or bracket notation (`obj['property']`), adding/modifying properties, deleting with delete keyword, checking existence with 'in' operator, getting keys (`Object.keys()`), values (`Object.values()`), entries (`Object.entries()`), cloning with spread operator (`...`), merging objects, iterating with for...in loop, and checking types with `typeof` and `instanceof`.

Q64. What are the common Array methods in JavaScript/TypeScript?

Answer: push() – adds to end; pop() – removes from end; unshift() – adds to beginning; shift() – removes from beginning; splice(index, count) – removes/adds at specific index; slice(start, end) – returns a shallow copy of a portion; indexOf() – finds first index of element; includes() – checks if array contains a value; forEach() – executes function for each element; map() – creates new array with transformed elements; filter() – creates new array with elements passing a test; sort() – sorts elements; concat() – merges arrays.

Q65. What are common String methods in JavaScript/TypeScript?

Answer: length – number of characters; charAt(index) – character at index; trim() – removes whitespace from both ends; replace(old, new) – replaces first occurrence; replaceAll() – replaces all occurrences; toUpperCase()/toLowerCase() – case conversion; substring(start, end) – extracts a section; split(separator) – splits into array; indexOf()/lastIndexOf() – finds substring position; startsWith()/endsWith() – checks beginning/end; includes() – checks if contains substring; padStart()/padEnd() – pads string to target length; match() – matches against regular expression.

Q66. How do you perform Data Conversions in JavaScript/TypeScript?

Answer: Any type to String: use String() constructor or .toString() method (concatenation with string also auto-converts). String to Integer: use parseInt() function. String to Float/Double: use parseFloat() function. You can use regular expressions to remove non-numeric characters before parsing. Generating unique IDs: convert date to string using toLocaleString().

Q67. What are Modules in TypeScript? Explain imports and exports.

Answer: A module is a file that contains code and explicitly exposes selected parts using export. Other files access them using import. Types: 1) Named Exports – export multiple values, import names must match exactly: export function add() {}; import { add } from './file'. 2) Default Export – one per file, import name can be anything: export default function multiply() {}; import multiply from './file'. 3) Import with Alias: import { add as sum } from './file'. 4) Namespace Import: import * as MathUtils from './file'. Configuration requires 'type':'module' in package.json and module settings in tsconfig.json.

Q68. Explain Exception Handling in JavaScript/TypeScript.

Answer: Exception handling uses try...catch...finally and throw. The try block contains code that might throw an exception. The catch block handles exceptions thrown within try. The finally block executes regardless of whether an exception occurred (used for cleanup). The throw statement throws custom exceptions: throw new Error('message'). Example: try { code } catch (error) { handle error } finally { always executes }.

Q69. What are Classes, Objects, and Constructors in TypeScript?

Answer: A Class is a structured way to define a blueprint for objects using the class keyword. A Class Object is an instance created using the new keyword. A Constructor is a special method that initializes object properties when the class is instantiated. Example: class


```
Employee { name: string; constructor() { this.name = 'Bharath'; } printName(): void {  
  console.log(this.name); } }. Create object: const emp = new Employee();
```

Q70. What are Access Modifiers in TypeScript?

Answer: Access modifiers control where class members can be accessed. Three types: public – accessible inside class, child class, and outside (default); protected – accessible inside class and child class, NOT outside; private – accessible only inside the class, NOT in child class or outside. Example: class Person { public name; protected age; private ssn; }

Q71. What is the difference between 'this' and 'super' keywords?

Answer: 'this' refers to the current object instance calling the method. It is used to access properties and methods of the current class. 'super' refers to the parent (base) class. It is used to call the parent constructor (mandatory in child class constructors) and to call parent methods. In method overriding, this.method() calls the child version while super.method() calls the parent version.

Q72. Explain Inheritance in TypeScript.

Answer: Inheritance is the mechanism by which one class can acquire properties and methods from another class using the extends keyword. The child class inherits all public and protected members of the parent class. Types: Single Inheritance – one child extends one parent. Multilevel Inheritance – a chain (Class3 extends Class2, Class2 extends Class1). Multiple Inheritance – not directly supported in TypeScript/JavaScript (a class cannot extend multiple classes).

Q73. Explain the four OOP concepts: Abstraction, Encapsulation, Polymorphism, and Inheritance.

Answer: Abstraction – hiding complexity by exposing only necessary details. Achieved using private properties/methods for implementation and public methods for interface. Encapsulation – bundling data and methods into a single unit, hiding internal state using private properties and exposing controlled access via public getter/setter methods. Polymorphism – includes Method Overriding (child class provides its own implementation of parent method with same name) and Method Overloading (same method name with different parameter signatures). Inheritance – one class acquiring properties/methods from another via extends keyword.

Q74. What are Static Properties and Methods in TypeScript?

Answer: Static members belong to the class itself, not to instances. They are accessed directly from the class without creating an object. Static methods and properties use the static keyword. Example: class Student { static rollNumber = 1234; city = 'Hyderabad'; static nameAndId() { return this.rollNumber; } }. Access: Student.rollNumber (no object needed) vs new Student().city (object required). Static members cannot be accessed from instances and vice versa.

Q75. Explain async/await, Promises, and Callback functions.

Answer: Callback Functions are functions passed as arguments to other functions and executed after an async operation completes. Promises represent the eventual completion or failure of an async operation, allowing you to attach handlers for success or failure. async/await provides elegant, synchronous-looking syntax for working with promises – async declares a function returns a promise, await pauses execution until a promise is settled. This progression (callbacks → promises → async/await) represents increasingly cleaner ways to handle asynchronous operations.